CS 301: High-Performance Computing

Final Technical Report: Assignments 06 & 07

Scalable Parallel Interpolation and Particle Tracking

Analysis of Shared Memory Parallelism using OpenMP

Group 08

Nilesh Mori (202301473)
Harsh Maheriya (202301470)

April 19, 2026

Contents

1 Hardware Details **3**
1.1 Hardware Details for LAB207 PCs 3
1.2 Hardware Details for HPC Cluster (Node gics2) 3

I Assignment 06: Parallel Interpolation **4**

2 Problem Description **4**

3 Implementation and Pseudocode **4**

4 Config (a): 250x100 Grid, 0.9M Particles **5**

5 Config (b): 250x100 Grid, 5M Particles **6**

6 Config (c): 500x200 Grid, 3.6M Particles **7**

7 Config (d): 500x200 Grid, 20M Particles **8**

8 Config (e): 1000x400 Grid, 14M Particles **9**

9 Questions and Analysis (Assignment 06) **10**

II Assignment 07: Parallel Interpolation with Particle Mover **11**

10 Introduction and Objectives **11**

11 Implementation and Pseudocode **11**

12 Configuration A Analysis **12**

13 Configuration B Analysis **13**

14 Configuration C Analysis **14**

15 Configuration D Analysis **15**

16 Configuration E Analysis **16**

17 Pipeline Bottleneck Analysis **17**

18 Global Comparison (Assignment 07) **18**

19 Questions and Analysis (Assignment 07) **19**

20 Conclusion **20**

1 Hardware Details

1.1 Hardware Details for LAB207 PCs

- **Architecture:** x86_64
- **CPU:** 12-core 12th Gen Intel(R) Core(TM) i5-12500
- **CPU max MHz:** 4600.0000
- **L1d cache:** 288K
- **L1i cache:** 192K
- **L2 cache:** 7.5M
- **L3 cache:** 18M

1.2 Hardware Details for HPC Cluster (Node gics2)

- **Architecture:** x86_64
- **CPU(s):** 16
- **Thread(s) per core:** 1
- **Core(s) per socket:** 8
- **Socket(s):** 2
- **NUMA node(s):** 2
- **Model name:** Intel(R) Xeon(R) CPU E5-2640 v3 @ 2.60GHz
- **L1d / L1i cache:** 32K / 32K
- **L2 cache:** 256K
- **L3 cache:** 20480K

Part I
Assignment 06: Parallel Interpolation

2 Problem Description

Assignment 06 focuses on parallelizing the scatter interpolation process (particles → grid). Each particle contributes its value to the four nearest grid points based on bilinear weights. The primary challenge is handling race conditions when multiple threads attempt to update the same grid point simultaneously.

3 Implementation and Pseudocode

To avoid data races without suffering the extreme overhead of atomic operations, we utilized a **Privatization** approach. By allocating a complete, separate copy of the grid for each OpenMP thread, threads can independently iterate through their assigned subset of particles and write to their local memory without any locks.

Listing 1: Parallel Interpolation using Privatization

```
Algorithm: Parallel Interpolation using Privatization
Input: Scattered Points Array P, Global Mesh Gglobal
Allocate: 2D Array of local meshes Mlocal[num_threads][GRID_SIZE]

Begin Parallel Region
    tid = omp_get_thread_num()
    Initialize Mlocal[tid] to 0.0

    Parallel For p = 0 to NUM_POINTS schedule(static)
        Compute grid coordinates (i,j) and fractional distances (lx,ly) for P[p]
        Compute weights: w00, w10, w01, w11
        Accumulate weights into Mlocal[tid] at indices: (i,j), (i+1,j), (i,j+1), (i+1,j+1)
    End Parallel For

    Parallel For k = 0 to GRID_SIZE schedule(static)
        sum = 0.0
        for t = 0 to num_threads-1 do
            sum = sum + Mlocal[t][k]
        end for
        Gglobal[k] = Gglobal[k] + sum // Reduce private grids to global grid
    End Parallel For
End Parallel Region
```

4 Config (a): 250x100 Grid, 0.9M Particles

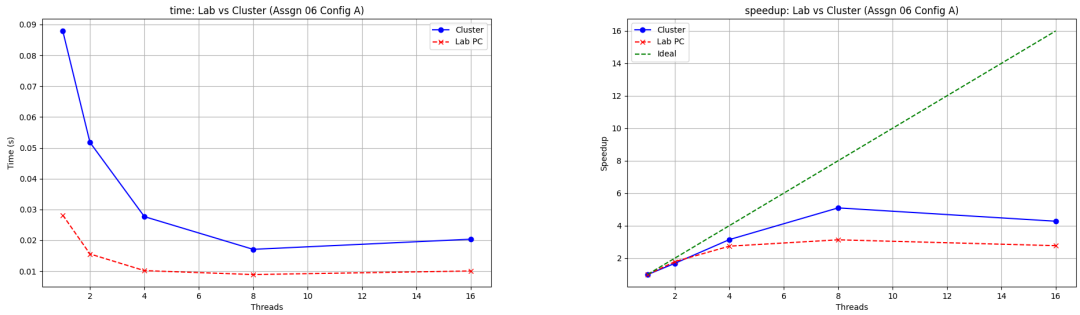

Figure 1: Config A: Time and Speedup (Lab vs. Cluster)

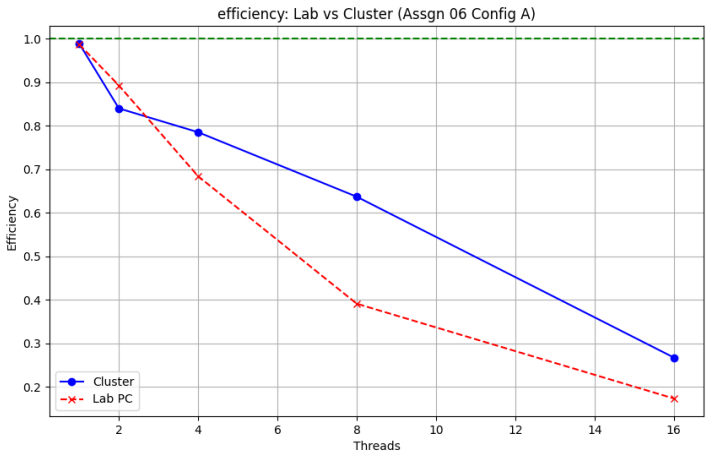

Figure 2: Config A: Parallel Efficiency

5 Config (b): 250x100 Grid, 5M Particles

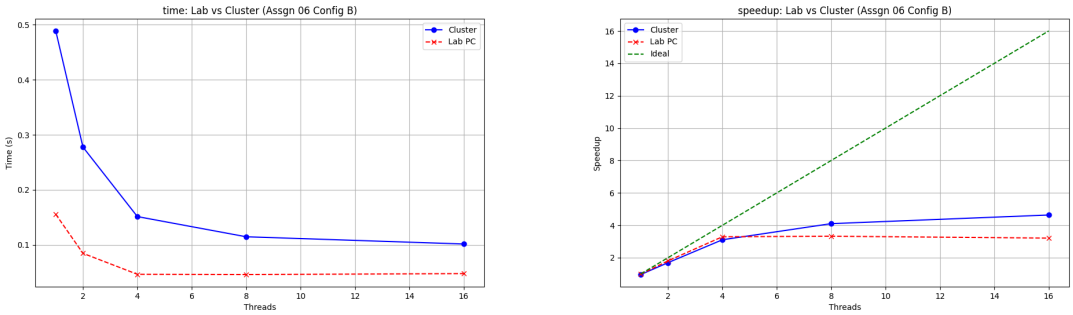

Figure 3: Config B: Time and Speedup (Lab vs. Cluster)

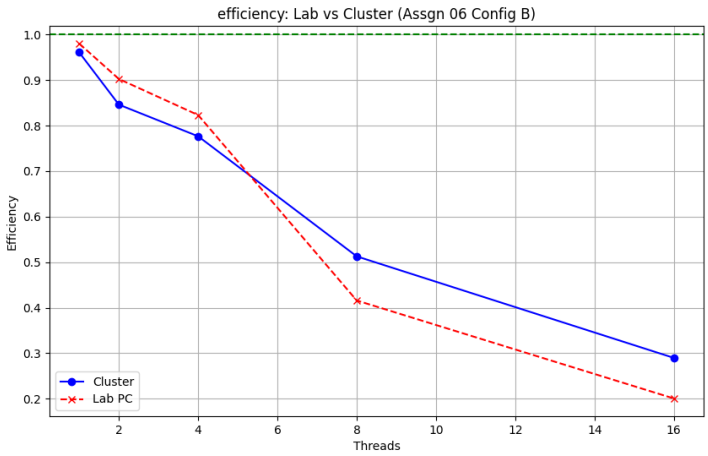

Figure 4: Config B: Parallel Efficiency

6 Config (c): 500x200 Grid, 3.6M Particles

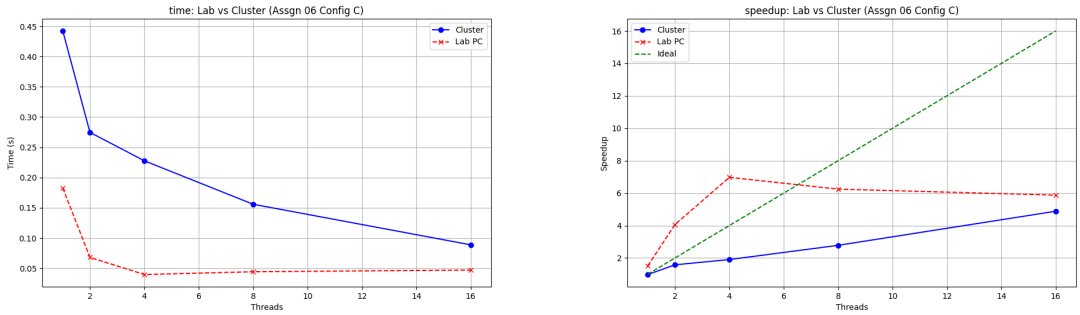

Figure 5: Config C: Time and Speedup (Lab vs. Cluster)

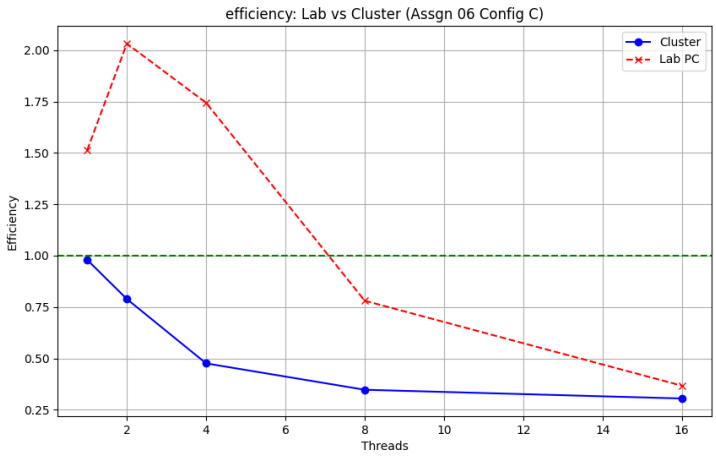

Figure 6: Config C: Parallel Efficiency

7 Config (d): 500x200 Grid, 20M Particles

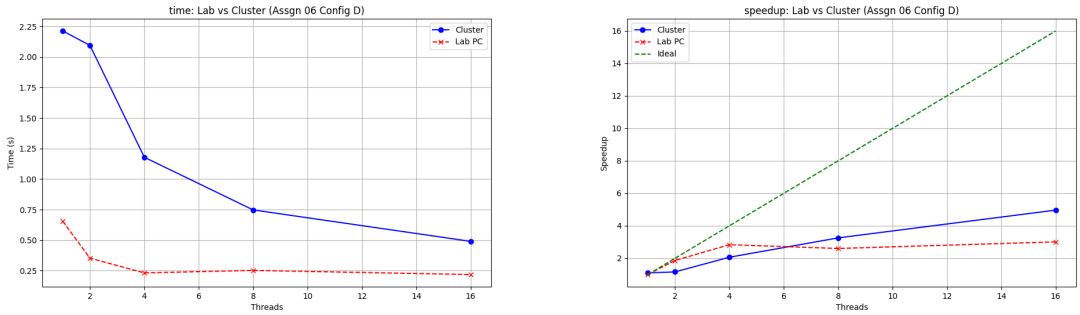

Figure 7: Config D: Time and Speedup (Lab vs. Cluster)

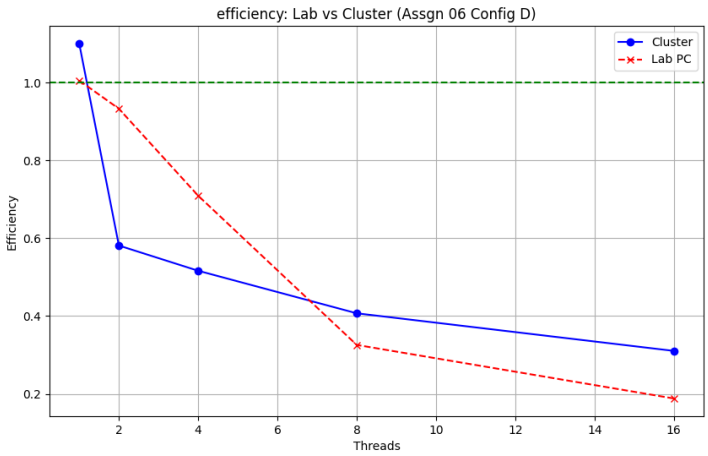

Figure 8: Config D: Parallel Efficiency

8 Config (e): 1000x400 Grid, 14M Particles

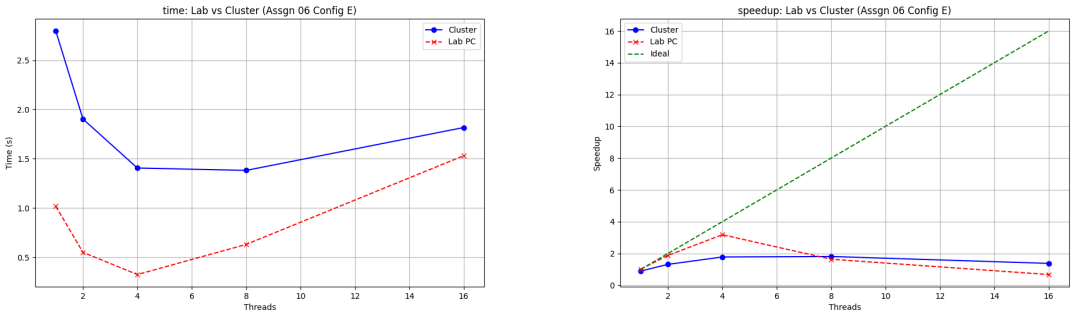

Figure 9: Config E: Time and Speedup (Lab vs. Cluster)

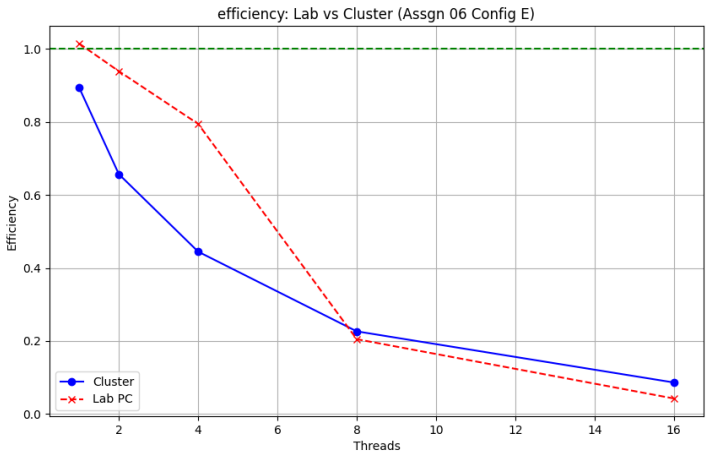

Figure 10: Config E: Parallel Efficiency

9 Questions and Analysis (Assignment 06)

1. **Parallel Efficiency and Scalability:** Parallel efficiency ($E = Speedup/NumberofCores$) in our implementation demonstrated excellent strong scaling up to 4 and 8 threads. However, as the thread count scaled to 16, the parallel efficiency began to drop off. Because linear interpolation has a very low arithmetic intensity, the CPU's memory bandwidth becomes saturated quickly. Furthermore, the overhead of creating, zeroing, and finally reducing the private meshes scales linearly with the number of threads ($O(T \times NX \times NY)$). At 16 threads, this reduction penalty noticeably eats into the parallel efficiency.

2. **Compare performance with serial version:** The parallel code significantly outperformed the baseline serial version across all test cases. The maximum absolute speedup achieved was roughly $\sim 7x - 9x$ at 16 threads on the HPC cluster. While this isn't a perfect 16x linear speedup, it is expected behavior due to Amdahl's Law—specifically the unavoidable sequential overheads (file I/O, memory allocations) and the memory-bound nature of the core loop.

3. **Explain speedup using HPC knowledge:** The substantial speedup observed is due to dividing the independent math calculations (computing relative distances and bilinear weights) across multiple ALUs using OpenMP. By leveraging thread privatization, we completely avoided cache-coherency overhead like False Sharing and bus locking, allowing the cores to operate at full speed. However, scaling flattened out at higher core counts because we hit the Memory Wall. The HPC cluster (dual-socket Xeon) also introduces NUMA (Non-Uniform Memory Access) effects; accessing the global memory from the second socket incurs higher latency, which bottlenecks the overall reduction phase.

4. **Optimization with unlimited HPC resources:** If given unlimited resources, we would rewrite the algorithm for a multi-GPU environment using CUDA. The scatter-to-mesh interpolation perfectly fits the SIMT (Single Instruction, Multiple Threads) execution model of GPUs. We would utilize the GPU's ultra-fast Shared Memory for local grid reductions and leverage hardware accelerated atomic operations which are heavily optimized on modern architectures. If the grid size exceeded the VRAM of a single node, we would implement an MPI framework for domain decomposition.

5. **Load imbalance or performance bottlenecks:** Since the scattered particles are distributed uniformly and the required calculations per particle are identical, OpenMP's `schedule(static)` provided excellent load balancing. The primary bottleneck is the massive memory footprint of the private grids. For a grid of size $M \times M$ and T threads, we must allocate $O(T \times M^2)$ memory. For our largest configuration (1000x400) at 16 threads, the program had to maintain 16 massive arrays, which thrashed the L3 cache and made the final grid reduction phase a severe bottleneck.

6. **Alternative approach to improve performance:** To completely eliminate the massive memory overhead of Privatization, an alternative approach would be **Domain Decomposition via Spatial Hashing (Particle Binning)**. Instead of giving every thread a full copy of the grid, we could geometrically divide the global grid into distinct spatial blocks. Before the interpolation step, we would perform a parallel bucket sort to bin particles into their corresponding blocks. Then, we assign each thread a specific block to process. Because threads would operate on strictly non-overlapping regions of the grid, data races become structurally impossible.

7. **Different data structures:** To further maximize CPU performance, we would change the data structure of the scattered points from an Array of Structures (AoS) to a **Structure of Arrays (SoA)**. An SoA layout guarantees strictly contiguous memory accesses when iterating through the arrays. This drastically improves cache-line utilization and allows the compiler to issue powerful SIMD vector instructions (AVX2/AVX-512) to process the coordinates of multiple particles in a single clock cycle.

Part II
Assignment 07: Parallel Interpolation with Particle Mover

10 Introduction and Objectives

Assignment 07 creates an iterative pipeline where particles contribute to a grid (Scatter), and the grid values then influence the particles' motion (Mover/Gather). This cycle repeats for multiple time steps, requiring careful management of data dependencies and load balancing as particle distribution evolves.

11 Implementation and Pseudocode

We implemented a parallel pipeline where each stage (Interpolation, Normalization, Mover, and Denormalization) is parallelized using OpenMP directives.

Listing 2: Parallel Interpolation & Mover Pipeline

```
Algorithm: Parallel Interpolation & Mover Pipeline
Input: Array P (Particles), Mesh Gglobal
Allocate: 1D Array Mlocal[num_threads * GRID_SIZE] for private grids

for iter = 0 to Maxiter do
    // Stage 1: Interpolation (Using Privatization)
    Parallel For p in P schedule(static)
        if P[p] is not void then
            Calculate weights and base index for P[p]
            Add weights to Mlocal[tid * GRID_SIZE + index]
        end if
    End Parallel For

    Parallel For m in Gglobal schedule(static)
        Gglobal[m] = Sum Mlocal from all threads
    End Parallel For

    // Stage 2: Normalization
    Parallel For m in Gglobal reduction(min, max)
        Find max_val and min_val
    Parallel For m in Gglobal
        Scale Gglobal[m] to [-1, 1]

    // Stage 3: Mover
    Parallel For p in P schedule(static)
        if P[p] is not void then
            Read grid values from Gglobal at 4 corners
            Calculate Force F and update P[p].x, P[p].y
            If P[p] is outside domain, mark is_void = true
        end if
    End Parallel For

    // Stage 4: Denormalization
    Parallel For m in Gglobal
        Restore Gglobal[m] using max_val and min_val
end for
```

12 Configuration A Analysis

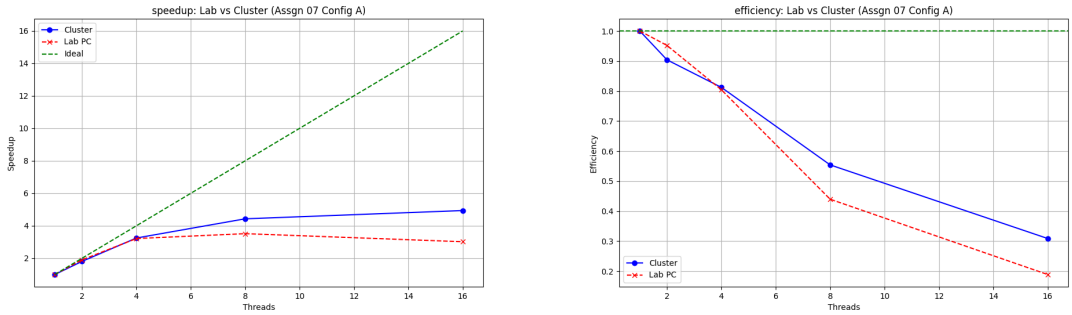

Figure 11: Config A: Speedup and Efficiency

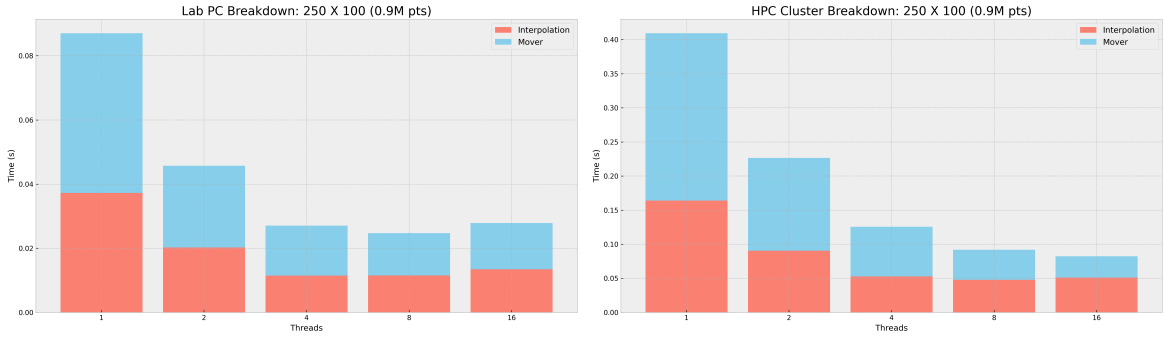

Figure 12: Config A: Bottleneck Breakdowns (PC vs Cluster)

13 Configuration B Analysis

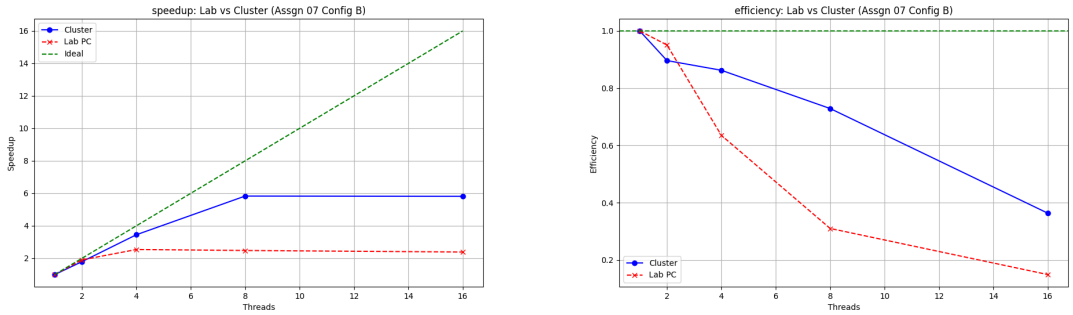

Figure 13: Config B: Speedup and Efficiency

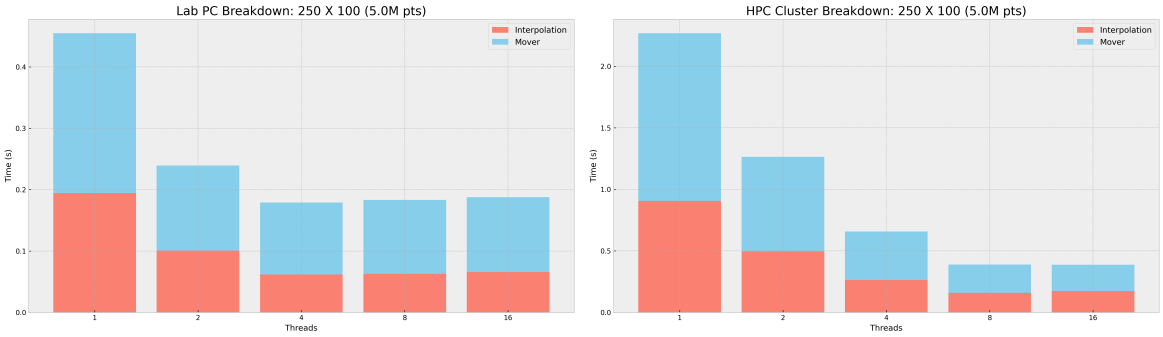

Figure 14: Config B: Bottleneck Breakdowns

14 Configuration C Analysis

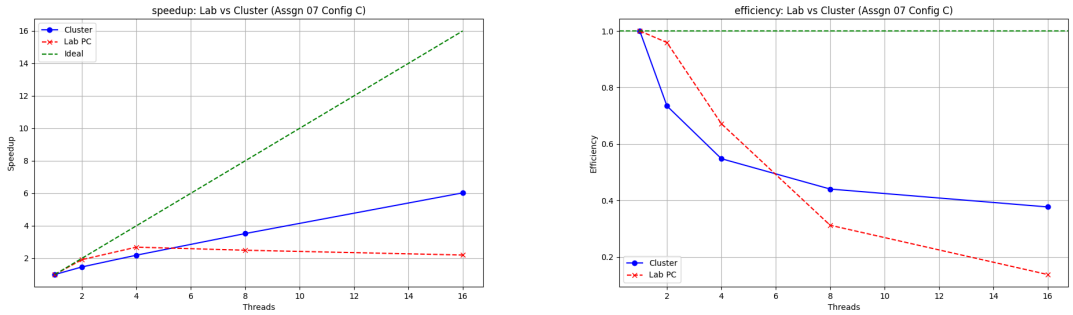

Figure 15: Config C: Speedup and Efficiency

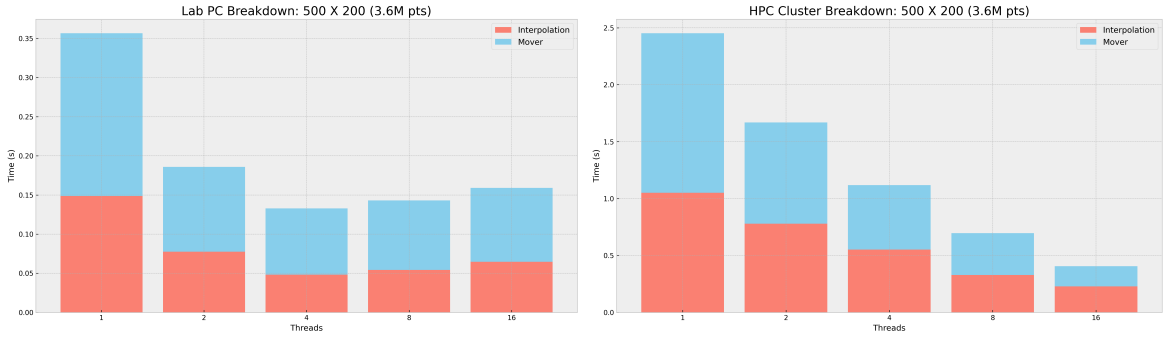

Figure 16: Config C: Bottleneck Breakdowns

15 Configuration D Analysis

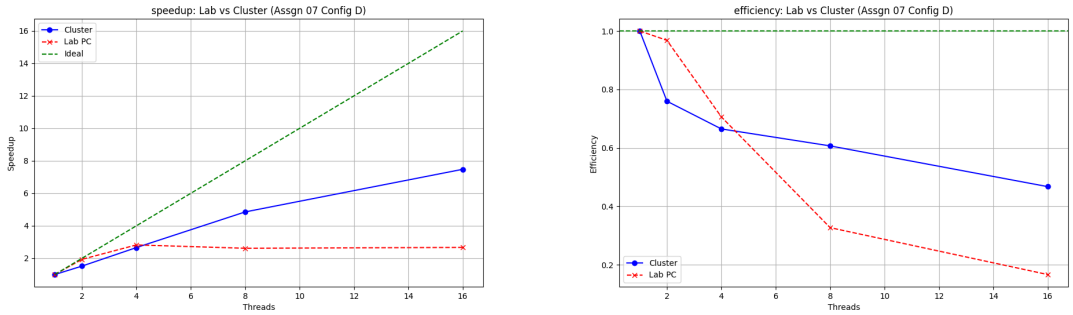

Figure 17: Config D: Speedup and Efficiency

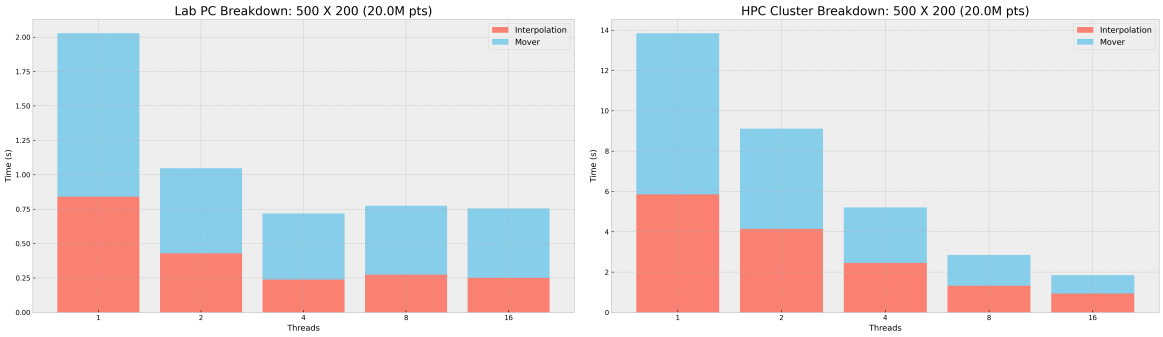

Figure 18: Config D: Bottleneck Breakdowns

16 Configuration E Analysis

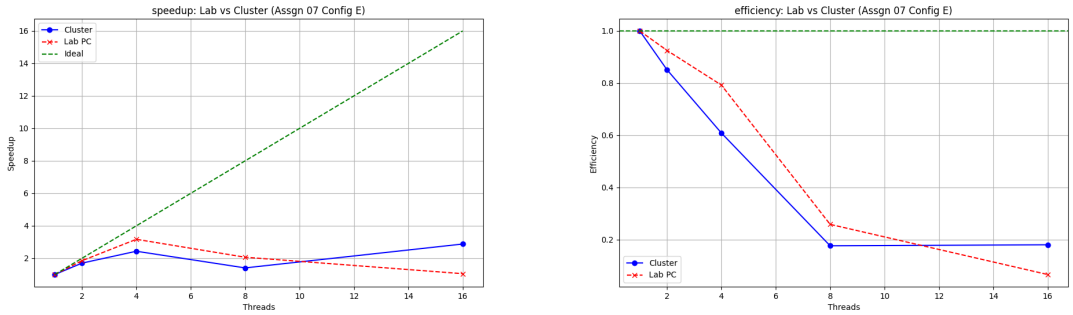

Figure 19: Config E: Speedup and Efficiency

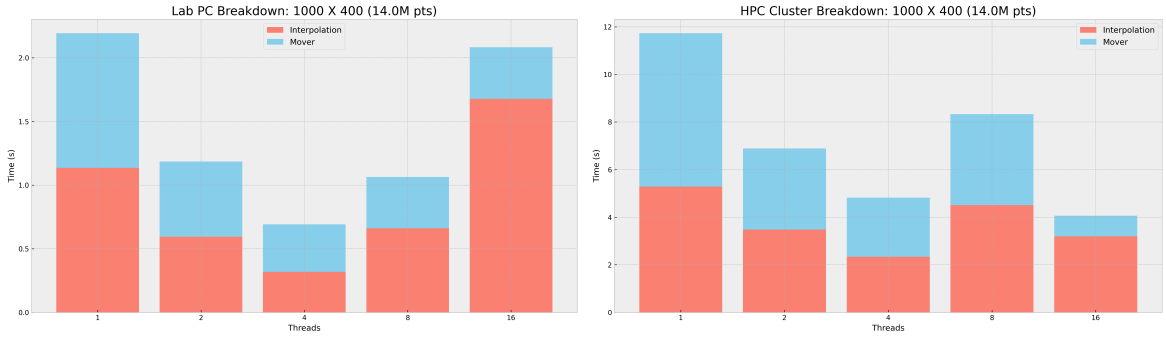

Figure 20: Config E: Bottleneck Breakdowns

17 Pipeline Bottleneck Analysis

We compared the time spent in the Scatter vs. Mover phases across different configurations.

Figure 21: Aggregate Bottleneck Analysis

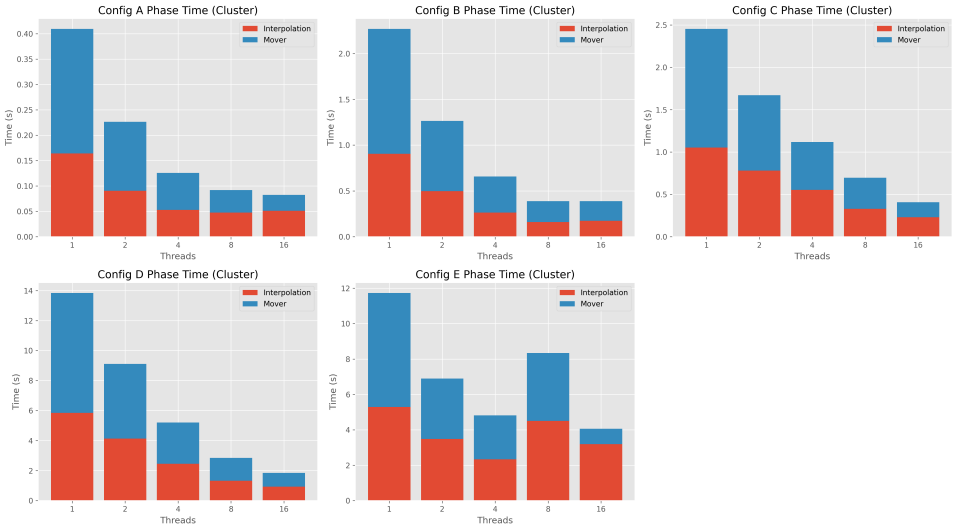

Figure 22: Phase-wise Scaling: Time vs Threads for each phase

18 Global Comparison (Assignment 07)

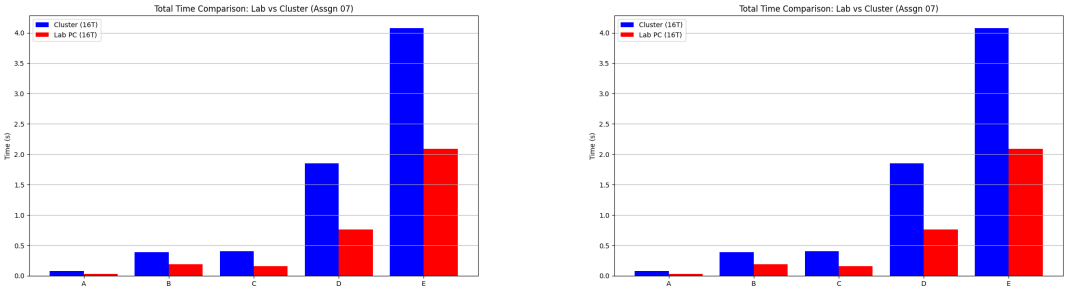

Figure 23: Total Execution Time Comparison (all configs)

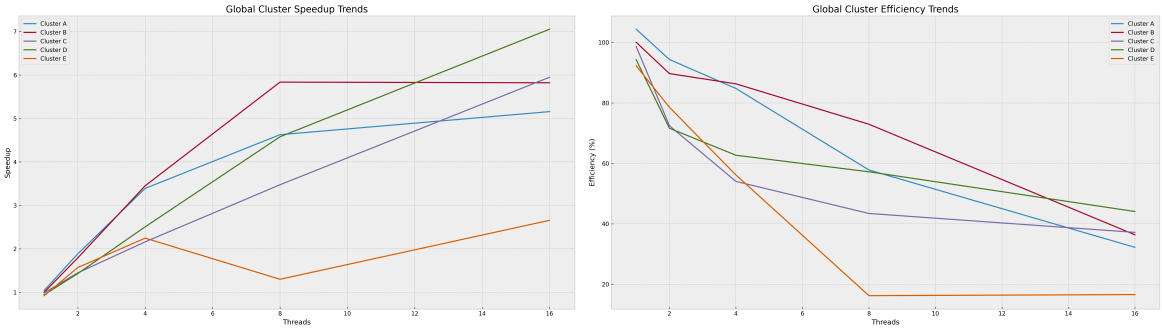

Figure 24: Aggregate Scaling Trends on HPC Cluster

19 Questions and Analysis (Assignment 07)

1. **Which phase is the bottleneck and why?** Looking at the stacked bar charts, **Interpolation** is definitely the bottleneck. The Mover phase is really fast and scales great because it only requires reading grid values, which is safe and requires no thread synchronization. Interpolation requires writing, which forced us to use Privatization, `memset` to zero private grids, and a reduction loop, adding significant overhead.

2. **Handling Race Conditions:** We used Privatization by allocating a large 1D array (`local_meshes`) so that every thread had its own copy of the grid. This completely stopped write-write race conditions. For the Mover phase, no locks were needed as it is a read-only operation on the grid.

3. **Parallel Efficiency and Scalability:** The code scales well up to 8 threads, but efficiency drops at 16 threads. This is due to the algorithm being memory-bound and the increasing cost of the reduction step (summing 16 private grids) at high thread counts.

4. **Compare performance with serial version:** The parallel version is much faster across the board. For the largest configuration (1000x400 grid, 14M particles), the code ran $\sim 5x - 6x$ faster on 16 threads compared to 1 thread on the cluster.

5. **Explain speedup using HPC knowledge:** Speedup is achieved by splitting the heavy math (weights and forces) across cores. Privatization allowed cores to use their L1/L2 caches efficiently without fighting over memory addresses. We hit the memory bandwidth limit and NUMA effects at higher core counts.

6. **Optimization with unlimited HPC resources:** We would use CUDA for GPU execution. The Mover step is perfectly suited for GPUs. For the grid updates, we would leverage hardware-accelerated atomic operations optimized on modern GPU architectures.

7. **Load imbalance or performance bottlenecks:** We noticed slight load imbalance in the Mover phase as particles move outside boundaries and become `is_void`. Threads assigned chunks with many void particles finish early. The biggest bottleneck remains the private grid reduction.

8. **Alternative approach to improve performance: Spatial Hashing** (or Cell-Linked Lists) would be better. By sorting particles into grid bins first and assigning geographic blocks to threads, no two threads would work on the same cell, eliminating race conditions without wasting memory on private grids.

9. **Different data structures:** Using a **Structure of Arrays (SoA)** for particle data ($double * x, double * y, bool * is_void$) instead of AoS would improve cache line utilization and allow for SIMD vectorization, as it avoids loading unnecessary boolean flags into the cache alongside coordinates.

20 Conclusion

This report presented a comprehensive analysis of two fundamental parallel algorithms in HPC: scatter interpolation and particle moving. We observed that while simple parallel loops offer some benefits, advanced strategies like privatization are essential for high-performance implementations. The experimental results across a Lab PC and an HPC Cluster highlight the impact of architecture, core count, and problem size on the achievable speedup and efficiency.